

LLM4Rec Homework Report — Assignment A02

From Item Similarity to User Profiles: Evaluating a Content-Based Movie Recommender

Student: Fengru Chen

Team: Individual

Email: fchen@edu.hse.ru

Date: 2026-09-22

Assignment: A02 — Content-Based Filtering (Week 2)

Abstract

This assignment implements and evaluates a content-based movie recommender on MovieLens 100K using cosine similarity and a three-movie user profile. I first corrected a genre-parsing defect, then implemented Top-5 item-to-item and profile-based retrieval, and finally tested watched-item leakage, profile averaging, popularity bias, and catalog discovery. After excluding every previously rated movie, 88.43% of users changed their ordered Top-5 between item and profile retrieval, while an IDF-weighted representation increased mean distinct profile similarity scores from 18.47 to 137.68. The main lesson is that, in this experiment, representation and evaluation design mattered more than simply replacing one similarity formula with another.

Index Terms — content-based filtering, cosine similarity, user profiles, MovieLens, IDF

1. Introduction

The Week 2 task is to implement cosine-based content similarity, construct a user profile by averaging multiple watched movies, and compare Top-5 item-to-item and profile-based recommendations. A movie is represented by a binary genre vector, and recommendation is based on similarity in this content space. Although the core algorithm is straightforward, its evaluation raises three broader questions: the trade-off between current-item intent and an aggregated profile, whether recommendations become popularity-skewed, and which method discovers more of the catalog. I therefore treated the assignment as two stages. First, I verified that the feature representation and similarity computation were correct. Second, I tested three failure modes from the class feedback: already-watched leakage, popularity skew, and profile-averaging failure.

The report contributes:

- a corrected and verified cosine-based item/profile recommender;
- a leakage-free user evaluation with popularity and catalog diagnostics;
- an IDF-weighted representation experiment to test whether it reduces coarse-feature ties.

Repository: <https://github.com/chenfengru/LLM4Rec-Course>.

2. Related Work

Lops et al. describe content-based recommendation as matching item representations to user preference profiles [1]. MovieLens 100K provides explicit user–movie ratings and timestamps used in the evaluation protocol of this assignment [2]. For the optional representation experiment, I adapted inverse-document frequency as a feature-weighting heuristic [3], treating movies as

documents and genres as binary terms.

I also considered Jaccard similarity and raw dot product. Jaccard measures set overlap, while raw dot product does not normalize vector magnitude. The required cosine method was therefore retained as the main implementation, and the alternatives were used only for diagnostics.

3. Method

The workflow was: baseline audit → feature-parser correction → cosine item retrieval → three-movie profile → failure-mode tests → IDF representation experiment.

3.1 Movie Representation and Cosine Similarity

The starter parser extracted all 19 values but supplied only 18 genre labels, causing a one-position semantic misalignment and dropping the final `Western` flag. I added the missing `unknown` label before comparing algorithms.

For movie vectors x and y ,

$$\cos(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}. \quad (1)$$

Zero-norm comparisons return score 0. The active movie is excluded and the five highest-scoring candidates are returned.

3.2 Three-Movie User Profile

For three recent positively rated movies,

$$p_u = \frac{x_1 + x_2 + x_3}{3}. \quad (2)$$

Candidates are ranked by $\cos(p_u, x_j)$. The most recent positive movie is also used as the item-to-item query, making the two modes directly comparable.

Because cosine similarity is invariant to positive scalar multiplication, using the sum instead of the average would produce the same ranking; the average is retained for an interpretable centroid representation.

For the controlled user evaluation, a user is eligible with at least three ratings ≥ 4 . All movies previously rated by that user are removed from the candidate set, so the evaluation enforces

$$\text{Top5} \cap \text{watched_set} = \emptyset. \quad (3)$$

3.3 IDF Representation Variant

Binary genre vectors produced many equal cosine scores. I therefore tested an IDF-weighted representation as a controlled feature-weighting variant:

$$\text{idf}_g = \log \frac{N + 1}{df_g + 1} + 1, \quad x'_{ig} = x_{ig} \text{idf}_g. \quad (4)$$

This gives rarer genres more weight. The experiment tests representation discriminability, not recommendation relevance.

3.4 Configuration

The experiments used MovieLens `u.item` and `u.data`, CPython 3.10.11, Node.js 24.19.0 from the Codex runtime, and a `1e-12` tie tolerance. Tie-sensitivity tests used fixed seeds 0–19 with one movie-priority mapping shared across compared methods. The head was defined as the top 20% of movies by rating count; the remaining 80% formed the long tail.

4. Experiments

4.1 Formula and Parser Verification

For *Toy Story* and *Aladdin*, the manually computed cosine was

$$\frac{3}{\sqrt{3} \cdot 2} = 0.8660254, \quad (5)$$

which matched both the Python audit and JavaScript implementation exactly.

The parser audit also confirmed that *Toy Story* is Animation/Children’s/Comedy and that the Western-only *Unforgiven* retains its `Western` feature after the repair.

4.2 Leakage and Item/Profile Results

The first user-level harness excluded only the three selected positive movies. This was incorrect: among 942 eligible users, 598 had at least one already-rated recommendation, for 1,621 leaked recommendation slots in total.

After changing the candidate filter to exclude each user’s complete rated history, all deterministic and 20-seed randomized checks passed with zero watched-item overlap.

Table 1: Corrected item-to-item versus profile evaluation

Metric	Item-to-item	Profile
Mean rating-count popularity	108.19	114.02
Median rating-count popularity	75.5	89.0
Long-tail share	58.41%	55.54%
Unique recommended movies	596	522
Catalog coverage	35.43%	31.03%

The ordered Top-5 changed for 833/942 users (88.43%), and the two methods shared fewer than one recommendation on average (0.926/5). Across 20 paired-random tie seeds, profile mean and median popularity were higher in 20/20 seeds and profile coverage was lower in 20/20. Item long-tail share was higher in 19/20 seeds.

4.3 Profile Averaging Diagnostic

Across 2,826 profile/input pairs, mean profile-to-input cosine was 0.720. The mean minimum similarity per user was 0.573 and the 10th-percentile minimum was 0.378.

A concrete minority-taste case occurred for user 104. The averaged profile had similarity 0.976 to both *Return of the Jedi* and *Star Wars*, but only 0.218 to the Comedy-only *Fierce Creatures*. This is evidence of minority-taste dilution in some profiles, not universal profile collapse.

4.4 IDF Results

IDF changed Top-5 membership for 704/942 profile queries (74.73%). This supports the narrower claim that the representation became more discriminative; it does not establish higher recommen-

Table 2: Binary versus IDF profile discriminability

Metric	Binary profile	IDF profile
Mean distinct scores	18.47	137.68
Top-5 contains tie	99.89%	90.87%
Cutoff tie	98.83%	85.99%
Membership-sensitive cutoff	92.68%	75.58%
Mean cutoff tie-group size	61.07	31.35

dation quality. Large tie groups still remained because movies with identical genre combinations remain identical even after IDF weighting.

5. Discussion

5.1 BUILD

I first ran the instructor baseline unchanged instead of immediately implementing the final solution. This revealed that the application worked, the application ran, but the genre mapping was semantically incorrect: 19 raw genre flags were aligned with only 18 labels.

After repairing that representation, I implemented the required cosine item-to-item Top-5 and three-movie profile Top-5 modes.

5.2 ASK WHY

Changing Jaccard to cosine seemed like it should strongly change the recommendations. However, only 1 of 1,682 catalog-level Top-5 lists changed. This suggested that the main limitation was not the similarity formula alone, but the coarse binary genre representation.

I then asked whether a correct cosine implementation was enough to make the user-level evaluation valid. This led to explicit checks for watched-item leakage, popularity skew, catalog coverage, and profile averaging.

5.3 TEST

I manually computed an item cosine and checked it against both Python and JavaScript. I also verified known genre mappings and reran the user evaluation after each candidate-filter change.

The most important test was

$$\text{Top5} \cap \text{watched_set} = \emptyset. \quad (6)$$

The historical harness failed this test. After correction, all deterministic checks and all 20 paired-random tie-breaking runs produced zero leaked recommendation slots.

5.4 EXPLAIN

The starter parser failed because the `unknown` genre label was missing, not because `u.item` contained malformed data. A simple schema assertion comparing the number of labels and raw fields would prevent the same error.

The user-level evaluation had a different root cause: it removed only the three movies used to form the profile, not the user's full rated history. The corrected filter therefore changed the candidate set rather than the similarity formula.

For the business questions, profile aggregation produced substantially different exposure but recommended more popular items on average and covered fewer unique catalog items in this

protocol. A popularity penalty, long-tail quota, or diversity-aware reranker could be tested later as bias mitigation, but none was evaluated here.

The averaging diagnostic also showed that a profile can preserve dominant tastes while weakening a minority taste. This is why a single centroid is not always a complete representation of a multi-interest user.

5.5 IMPROVE

The final controlled improvement was IDF weighting of genre features. It increased the number of distinct similarity scores and reduced large profile tie groups without changing the underlying content-based framework.

With more time, I would keep the watched-item leakage and profile-averaging checks as regression tests, then evaluate richer textual or learned content representations using held-out positive interactions. This would test recommendation relevance rather than only score discriminability.

6. AI Usage Disclosure

AI tools used. Codex CLI 0.155.0-alpha.9.2 was used for the formal coding and analysis workflow. ChatGPT (GPT-5.6 Sol) was used for conceptual clarification, experiment-review discussion, and report organization and wording. The exact underlying model version used by Codex was not exposed.

How AI was used. AI assisted with repository inspection, debugging hypotheses, implementation scaffolding, analysis scripts, browser-oriented UI changes, literature discovery, and report drafting.

What I personally verified. I ran the baseline and final application, inspected the genre mapping, hand-computed cosine values, checked sum/average profile behavior, reproduced the leakage failure, and compared key numerical claims against rerun analysis outputs.

What I trusted without full independent verification. I did not independently audit every internal AI tool call or every line of non-critical code organization. Critical algorithmic, mathematical, and benchmark claims were not accepted solely from AI output.

Session notes. The submitted `session.json` is the unmodified Codex audit trail for the formal Week 2 coding workflow. ChatGPT was used separately for conceptual tutoring and report review and is therefore not contained in that Codex log.

References

- [1] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in *Recommender Systems Handbook*, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. Boston, MA, USA: Springer, 2011, pp. 73–105. [Online]. Available: https://doi.org/10.1007/978-0-387-85820-3_3. [Accessed: Sep. 22, 2026].
- [2] F. M. Harper and J. A. Konstan, “The MovieLens datasets: History and context,” *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, Art. no. 19, 2015. [Online]. Available: <https://doi.org/10.1145/2827872>. Official PDF: <https://files.grouplens.org/papers/harper-tiis2015.pdf>. [Accessed: Sep. 22, 2026].
- [3] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988. [Online]. Available: [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0). [Accessed: Sep. 22, 2026].